Mi Red: clasificación de incidentes de red mediante NLP

Informe técnico del proyecto integrador de Tópicos y Programación Avanzada. Fecha de revisión: 2 de noviembre de 2026. Referencia del código publicado: commit 772a70d; este informe y sus pruebas se prepararon posteriormente, en el árbol de trabajo local.

1. Problema y alcance

Mi Red recibe descripciones escritas de incidentes, predice una categoría y permite consultar el procesamiento del texto. Un ticket relaciona la descripción con un dispositivo y un usuario. La API puede guardar la categoría, la probabilidad y el tiempo de inferencia, y cambiar el estado del ticket de PENDIENTE_DIAGNOSTICO a DIAGNOSTICADO. La predicción ayuda a clasificar reportes; no comprueba físicamente la causa de una falla.

El sistema implementado usa Django REST Framework, React con TypeScript y PostgreSQL alojado en Supabase. No captura tráfico, no ejecuta Cisco Packet Tracer ni corrige automáticamente dispositivos. Bonita BPM permanece como integración pendiente y su sección está oculta. Los reportes con Groq o DeepSeek son una función complementaria: redactan respuestas a partir de métricas calculadas por el backend y no sustituyen al clasificador entrenado.

2. Dataset y análisis exploratorio

Fuente declarada: https://huggingface.co/datasets/codingkiddo/smartnet-network-incidents. La ficha observada indicaba datos sintéticos en inglés y licencia Apache-2.0. La licencia y la revisión exacta del repositorio original deben comprobarse y archivarse antes de la entrega. La preparación local se realizó el 1 de noviembre de 2026; no se conserva evidencia independiente de la fecha exacta de descarga.

Se reunieron 6000 filas de las particiones descargadas. Se tradujeron doce plantillas de redacción con asistencia de IA, conservando cifras y unidades, y se mapearon las etiquetas al español. Se eliminaron 2854 repeticiones según la normalización del proyecto. El archivo resultante, data/raw/incidentes_es.csv, tiene 3146 filas y dos columnas: texto y categoria. Aunque se guarda en raw por decisión del equipo, es una versión traducida y depurada. Los originales y los programas de preparación se eliminaron a petición del usuario, lo que limita la reproducibilidad de esa preparación.

Categoría	Registros
LATENCIA_ALTA	994
RIESGO_SEGURIDAD_DNS	857
FUNCIONAMIENTO_NORMAL	510
CONGESTION_WIFI	365
SENAL_DEBIL	308
FALLO_DHCP	112
Total	3146

El notebook notebooks/01_exploracion_y_experimentos.ipynb incluye distribución de clases, longitudes, términos frecuentes, valores vacíos, duplicados y experimentos. El cargador rechaza textos vacíos, etiquetas vacías y duplicados normalizados. La clase mayoritaria tiene casi nueve veces los ejemplos de la menor. Se usa estratificación y F1 macro para dar igual importancia a las seis clases. En regresión logística se compara el uso de class_weight='balanced' frente a no ponderar.

SHA-256 del CSV utilizado: f88f7eeedf612a04deb0befb1cc78101c9ea18eaa6a272c44293868fdc178507. La nueva partición conserva 2516 filas para entrenamiento y 630 para prueba, con semilla 42. Se descartó la separación original de Hugging Face. Las variantes de las mismas plantillas aparecen en ambos conjuntos, por lo que esta prueba es poco exigente y no demuestra desempeño con incidentes reales.

3. Decisiones del pipeline NLP

La implementación se encuentra en src/nlp. El pipeline serializado incluye el preprocesamiento, CountVectorizer, TfidfTransformer y el clasificador. La API carga ese mismo objeto y no vuelve a entrenar ni ajusta su vocabulario con la consulta del usuario.

Normalizacion.py convierte a minúsculas y normaliza Unicode NFC. Conserva tildes y números porque pueden representar nombres, tiempos, canales o códigos. Sustituye URLs por enlaceweb y elimina signos y emojis. Por ejemplo, «¡SIN señal! canal 6 https://ejemplo.com [emoji]» se convierte en «sin señal canal 6 enlaceweb». Esta decisión simplifica el vocabulario, pero elimina la puntuación de direcciones IP y términos como Wi-Fi.

Tokenizacion.py separa por espacios el texto normalizado. Por ejemplo, «sin señal canal 6» produce [sin, señal, canal, 6]. Una dirección 192.168.1.1 se separa en fragmentos numéricos al normalizarla. Es una limitación reconocida que conviene mejorar si se incorporan reportes técnicos reales.

Stopwords.py usa palabras funcionales del español de spaCy y permite personalizar la lista. Conserva no, ni, nunca, sin y tampoco para evitar borrar negaciones relevantes. En «el equipo no conecta», se elimina «el» y se conserva «no». Mantener la negación no garantiza que un clasificador basado en bolsa de palabras comprenda su alcance.

StemmingLematizacion.py implementa SnowballStemmer en español y lematización con es_core_news_sm de spaCy. El stemming reduce formas como «conectando» a una raíz; la lematización busca una forma de diccionario según el contexto. Ambos se comparan en validación cruzada. Los resultados y ejemplos ejecutables se encuentran en el notebook; la configuración seleccionada utiliza stemming.

N_gramas.py compara unigramas con unigramas más bigramas. Para [sin, señal, wifi], los bigramas incluyen «sin señal» y «señal wifi». El vectorizador usa min_df=1, max_df=1.0 y max_features=10000, tokenización por espacios y sin convertir otra vez a minúsculas. El límite de 10000 es un máximo, no el tamaño real del vocabulario aprendido.

TF_IDF.py pondera la frecuencia de términos por su frecuencia inversa entre documentos. Usa norm='l2', use_idf=True, smooth_idf=True y sublinear_tf=False. El vocabulario y los pesos IDF se ajustan exclusivamente durante entrenamiento; en validación se ajustan dentro de cada partición mediante el pipeline de scikit-learn.

4. Modelos y experimentos

La tarea es clasificación supervisada multiclase. La regresión logística aprende pesos por característica y estima probabilidades de clase. Naive Bayes multinomial combina frecuencias de características con una hipótesis de independencia condicional. Ambos funcionan con matrices dispersas de TF-IDF y ofrecen una comparación sencilla y explicable.

Se utiliza GridSearchCV con tres particiones estratificadas, mezcla y semilla 42, optimizando F1 macro. Se prueban dos métodos morfológicos y dos rangos de n-gramas. La regresión logística compara C en {0.1, 1,

10} y ponderación None/balanced, sumando 24 configuraciones. Naive Bayes compara alpha en {0.1, 1, 5}, sumando 12. Los 36 candidatos producen 108 ajustes de validación, además de los reajustes finales. El test reservado se consulta después de elegir los parámetros.

Modelo	F1 macro CV	Accuracy test	Precisión macro	Recall macro	F1 macro test
Regresión logística	1.000	1.000	1.000	1.000	1.000
Naive Bayes	1.000	1.000	1.000	1.000	1.000

Se conserva regresión logística con C=0.1, class_weight='balanced', stemming y unigramas. Naive Bayes selecciona alpha=0.1, stemming y unigramas. La elección conserva el primer modelo ante un empate; estos resultados no demuestran que regresión logística sea superior. artifacts/metricas.json guarda todos los candidatos, sus parámetros, índices de la partición, versiones y métricas.

La precisión indica qué proporción de las predicciones de una clase son correctas. Recall indica qué proporción de los ejemplos de esa clase fueron recuperados. F1 combina ambas medidas y su promedio macro asigna el mismo peso a cada categoría. La matriz de confusión muestra etiquetas reales en filas y predichas en columnas.

5. Análisis de errores y límites

Se evaluó el pipeline ganador, sin reentrenarlo, sobre 18 descripciones adicionales sintéticas redactadas con asistencia de IA, tres por categoría. Sus etiquetas están pendientes de revisión técnica humana. Se obtuvieron 10 aciertos y 8 errores: accuracy 55.6 %, precisión macro 52.8 %, recall macro 55.6 % y F1 macro 52.2 %. Los valores completos están en artifacts/analisis_errores.json y las entradas en data/evaluacion/casos_revision.csv.

Hay confusiones entre latencia y DHCP al describir tiempos de respuesta con redacciones nuevas. Un caso de congestión con buena señal se clasifica como señal débil. También fallan descripciones de funcionamiento normal que contienen vocabulario de averías negadas. Son hipótesis coherentes con la baja variedad de redacción y con la representación por palabras; no prueban una causa única.

El 100 % y el 55.6 % corresponden a muestras distintas. Su diferencia no demuestra una degradación temporal ni estima el rendimiento en producción. Hace falta una evaluación mayor, con reportes independientes revisados por técnicos y separación por familias de escenarios. La probabilidad del modelo tampoco debe presentarse como certeza técnica ni como probabilidad calibrada de una causa física.

6. Arquitectura e integración

React envía las consultas autenticadas al backend. Django valida la entrada, carga artifacts/pipeline.joblib y obtiene categoría y probabilidad. La vista NLP muestra el resultado y el desglose del preprocesamiento. La animación secuencial del frontend es una presentación visual del resultado recibido, no telemetría en tiempo real de cada transformación.

La creación de tickets guarda descripción, dispositivo, autor y estado pendiente. POST /api/v1/nlp/diagnosticar/ recibe ticket_id, clasifica su descripción y realiza una actualización condicional para evitar sobrescribir un ticket modificado durante la inferencia. Guarda categoría, probabilidad, tiempo y fecha, y pasa a DIAGNOSTICADO. No calcula prioridad. La consulta NLP libre solo clasifica texto y no modifica tickets. La invocación del diagnóstico se puede demostrar por API; falta integrarla como acción visible en la pantalla de tickets.

Endpoint	Función
POST /api/v1/nlp/predecir/	Clase, probabilidad y preprocesamiento
POST /api/v1/nlp/diagnosticar/	Guarda el diagnóstico del ticket
GET /api/v1/nlp/metricas/	Comparación guardada y evaluación adicional
GET /api/v1/nlp/estado/	Estado del servicio y del pipeline
GET /api/v1/reports/metrics	Métricas calculadas con PostgreSQL

Las entradas de predicción deben tener de 10 a 2000 caracteres. Los endpoints requieren JWT. Los registros HTTP y de diagnóstico permiten auditar operaciones. El endpoint de estado verifica el pipeline, no la disponibilidad integral de PostgreSQL ni de proveedores externos.

7. Reproducibilidad y verificación

Desde la raíz, python -m src.models.entrenar --datos data/raw/incidentes_es.csv genera el pipeline y metricas.json. python -m src.models.analizar_errores produce la evaluación adicional. El artefacto registra scikit-learn 1.7.2, NLTK 3.9.2, spaCy 3.8.7, joblib 1.6.0 y es_core_news_sm 3.8.0. El entorno de análisis tiene requisitos separados. El backend todavía usa algunos rangos de dependencias, por lo que queda pendiente fijar completamente ese entorno.

Se ejecutaron 7 pruebas de NLP/modelos, 24 pruebas de backend y la compilación TypeScript/Vite. El registro docs/prueba_completa.md describe comandos, resultado de la integración real, alcance y comprobaciones manuales pendientes. La evidencia de ejecución es distinta de las métricas de calidad del clasificador. La instalación desde una máquina limpia requiere disponer del esquema externo de Supabase: sus tablas de negocio no son creadas por las migraciones Django y falta empaquetar ese aprovisionamiento.

8. Requisitos de entrega y trabajo del equipo

El repositorio contiene código, pipeline, notebook, README y CONTRIBUCIONES.md. Este documento cubre problema, dataset, NLP, comparación y arquitectura. Las diapositivas y el guion de demostración acompañan el informe. Aún deben verificarse la licencia original, la revisión de etiquetas, el acceso del docente y los pull requests revisados. Cada integrante debe mostrar un aporte significativo propio en NLP/ML y defenderlo; no se atribuyen aportes que no consten en Git. La etiqueta entrega-final y la publicación de estos archivos requieren el cierre del equipo.

9. Uso de IA y referencias

Se utilizó Codex como asistente para implementar, revisar y documentar código, preparar traducciones de plantillas, elaborar casos adicionales y generar estos materiales. Los casos adicionales y las traducciones requieren revisión humana. Las pruebas registradas fueron ejecutadas; las tareas manuales pendientes no se presentan como realizadas. Cada integrante debe entender y poder modificar el código que presenta.

Referencias de evidencia: Proyecto_Integrador.pdf, apartados 5 a 10; data/README.md; notebooks/01_exploracion_y_experimentos.ipynb; artifacts/metricas.json; artifacts/analisis_errores.json; docs/api_diagnostico.md; README.md; CONTRIBUCIONES.md e historial Git. Las bibliotecas se usan mediante sus API: scikit-learn, NLTK, spaCy, Django REST Framework y React. El informe describe la implementación disponible y sus límites; no certifica cumplimiento total de la evaluación individual.